

PRIVACY POLICY

=====

Document status: Active proof of concept and development state.

DevBox already has a functioning local launcher, a graphical main interface, modular structure views, master-data and documentation maintenance, documentation snapshots, local tool detection, a repository page, and initial building blocks for the DevBox-specific publication process.

The deskNode area is actively integrated into DevBox. Supervisor and daemon processes can be started and stopped from the GUI, and their console output is displayed in the deskNode log. The product version can be edited through master data. UX themes can be named, created, deleted, duplicated, renamed, and saved. Saving a theme refreshes the deskNode manufacturer database.

Symbol management now exists as an initial functional catalog interface. It manages deskNode device categories and consumer devices through selection menus and dialogs for creation, editing, and deletion. New devices receive one PNG source, stored under their record ID as `symbol_source_<record_id>.png`. After every successful catalog change, `create_manufacturer_db.py` is executed so `mnfctr_db.r0b` contains current theme, category, and device data.

The graphics-pack build is functional as a proof of concept. It creates multiple prepared consumer-graphics states from symbol sources and UX themes. Actual deskNode runtime integration, asset selection, language packages, and complete state logic are still being developed.

DevBox is not a finished end-user or release version. Interfaces, database migrations, product modules, publication workflows, document templates, and tests are continuously being revised. This documentation describes the current state and must be updated alongside functional changes.

First publication: Responsible party / publisher: Markus Walloner Name: Markus Walloner Country: Germany (DE)

1. Subject Matter

This privacy policy applies to the software or project described in the accompanying project documentation.

Short Description: DevBox is the local development hub of the CYXnTrol Development Platform. Its PySide6 interface brings together project structure, master data, documentation, local tools, product modules, repository preparation, and recurring development workflows.

DevBox is not an end-user product. It is an internal working environment in which technical requirements, data models, interfaces, build steps, and publication states are prepared, reviewed, and refined in a traceable manner.

2. Privacy Policy

PRIVACY POLICY – DRAFT VERSION

1. Purpose and scope

This privacy notice describes the currently intended local operation of DevBox. DevBox is a local development environment. Under its current concept, it is not designed to use telemetry, advertising, analytics services, or proprietary cloud services without explicit user action.

2. Controller and contact

The controller for the DevBox development state is Markus Walloner within CYX Labs. Questions about this project can be sent through the contact channel stated in the relevant repository or accompanying project documentation.

3. Locally processed data

DevBox processes the following data locally in particular:

- paths to the project root, project files, and local tools;
- contents of local SQLite databases, including product, manufacturer, documentation, repository, structure, UX-theme, category, and consumer metadata;
- local PNG sources for consumer devices, derived deskNode manufacturer databases, and prepared graphics packages;
- runtime and error logs in logfile.r0b;
- detected paths to Inkscape and GIMP in locdata.r0b;
- version numbers, theme names, RGBA values, category names, device keys, and further surface settings entered by the user;
- commit text as well as image paths and image files selected for explicitly triggered repository operations;
- technical output from local subprocesses, for example Python, Git, Inkscape, GIMP, or product-specific scripts.

This data is generally processed locally in project folders, temporary workspaces, or under AppData and is used to perform the DevBox function selected by the user.

4. Temporary workspaces

For documentation exports, graphics builds, repository preparation, and similar operations, DevBox creates unique temporary work folders. Copies of project files, intermediate graphics, masks, generated documents, and prepared assets may be processed there. The application attempts to remove these workspaces after completion. Failed cleanup may result in individual temporary files remaining until a later manual or system cleanup.

5. External programs

DevBox may start local programs such as Inkscape, GIMP, Git, Git Credential Manager, or installers after explicit user action. Once such a program starts, its own data processing and terms may additionally apply. DevBox does not store Git passwords or access tokens as its own repository credentials.

6. Repository operations and data transfer

When a push-to-Git operation is explicitly triggered, Git transmits the repository content selected by the user, commit information, and technically required connection data to the relevant repository provider. The specific recipient, repository visibility, and selected branch result from the repository configuration maintained by the user. The user is responsible for not publishing personal or confidential data unintentionally.

7. Legal basis and review

This text is a technical draft and not individual legal advice. Before public or commercial distribution, it must be reviewed and adapted to actual data flows, contact details, potential legal bases, processor arrangements, international transfers, and applicable data-protection law.

3. Project Context

Purpose: DevBox is intended to turn recurring and error-prone development work into traceable local tools. This includes maintaining product and manufacturer data, structured documentation, prepared publication steps, integration of local creative applications, and controlled execution of product-specific development functions.

The application provides a shared working foundation for products of the CYXnTrol Development Platform. Instead of keeping workflows only as memory, folder conventions, or a collection of console commands, they are retained as data, scripts, GUI functions, and verifiable process chains.

For deskNode, DevBox additionally serves as a development and configuration interface for product versions, UX themes, device categories, consumer symbols, and prepared graphics packages. The later deskNode runtime is intended to receive reproducible data and pre-rendered assets from this process.

Configuration: Configuration is separated into several layers.

Project root: The project root is discovered through the .root file. It is the authoritative source for scripts, resources, form archives, installers, global fonts, and the central DevBox database.

Central master data: resources/organization/devbox_db.r0b contains manufacturer, product, documentation, structure, repository, UX-theme, and symbol-catalog information. The "ux-deskNode" table contains multiple named theme records with record_id and theme_name. The desknode_consumer_device_categories and desknode_consumer_devices tables contain permanent technical categories and consumer devices. Schema extensions are intended to preserve existing records and add missing

fields through migration.

Symbol catalog: Categories are maintained through `category_key` and an automatically derived `translation_key`. Devices are maintained through `device_key`, `category_id`, and an automatically derived `translation_key`. When a device is created, one PNG is selected or dropped and copied to `resources/graphics/symbol_source_<record_id>.png`. Category association is stored through `category_id`, not through a file name.

deskNode runtime data: `applications/deskNode/data/mnfctr_db.r0b` is recreated by `create_manufacturer_db.py`. It contains `master_data`, `ux_themes`, `consumer_device_categories`, and `consumer_devices`. This refresh is executed after every successful change to version, theme, category, or consumer device.

Surface design: Colors are stored as eight-character RGBA hexadecimal values without `#`, for example `00e4ffff`. Font files are read recursively from `resources/fonts` and stored as project-root-relative paths. Font roles include large headings, section headings, body text, button text, input text, status messages, and log text. Additional theme values cover font size, font style, underlining, outlines, radii, and state colors.

Local runtime data: `%appdata%/CYX Labs/CYXnTrol/DevBox/logfile.r0b` contains log data. `%appdata%/CYX Labs/CYXnTrol/DevBox/locdata.r0b` contains verified paths for Inkscape and GIMP.

Tool detection: Stored paths are checked at startup. Invalid paths are searched again, first under Program Files and then in the system temporary area. Missing tools limit only the related functions.

Repository data: Repository URL and branch are maintained per product in `product_credentials`. Git credentials are not stored in DevBox. For the DevBox push, a temporary publication root is created; its applications directory is cleaned and then selectively filled with `applications/deskNode/resources/scripts` without `__pycache__` content.

Technology: DevBox currently uses the following technologies in particular:

- Python as the core language for launchers, process logic, data preparation, automation, and product-specific tools.
- PySide6 for the local graphical user interface.
- SQLite for central project master data, deskNode manufacturer data, and local runtime, log, and location data.
- openpyxl as a transitional or import tool for spreadsheets, not as the central master-data source.
- ReportLab and svglib for local generation of designed PDF documents.
- Git and Git Credential Manager for repository operations and authentication.
- Inkscape for scaling, SVG work, and vectorization of masks.
- GIMP 3 for non-interactive image processing and preparatory mask stages.
- XML/SVG processing through Python standard libraries for cleanup of generated masks.

- C# generation, the .NET SDK, WiX, and later additional packaging tools for build and installer processes.
- temporary workspaces under the system temporary directory for controlled copies, graphics builds, exports, and publication preparation.

The technical structure aims to clearly separate GUI, function scripts, subscripts, data sources, product databases, external tools, and temporary work states. The deskNode symbol catalog links SQLite records to PNG sources through stable record_id values, while the graphics-pack build creates the resulting visual variants.

Repository Note: The DevBox repository represents the CYXnTrol Development Platform itself, not a finished end-user product. It serves as a development record, a transparent work sample, and a foundation for maintaining the platform.

The published state is not copied directly from the productive project root. DevBox creates a separate cleaned root_dir state. It contains intended source code, documents, and assets, while local runtime data, temporary remnants, installers, unnecessary build output, and unpublished data remain outside the publication package.

The temporary publication root treats applications as a deliberate delivery window: the directory is cleaned first. It is then populated with applications/deskNode/resources/scripts and all contained files. __pycache__ directories as well as .pyc and .pyo files remain excluded. The real local product directory is not changed by this process.

DevBox is developed in an AI-assisted iterative workflow. Requirements, architecture, data models, UX decisions, test cases, and acceptance are directed by the project owner. Implementation is created with AI-assisted development tools and checked against defined functional requirements.

Copyright (c) 2026 Markus Walloner